

---

# Sync & Secrets CLI

rclone, rsync, Syncthing, Proton Drive CLI & Proton Pass CLI — five tools, one reference.

GitHub ↗

---

## TABLE OF CONTENTS

Which Tool for the Job

rclone — Remotes & Config

rclone — Copy, Sync & Move

rclone — Bisync (Two-Way)

rclone — Mount & Filters

rsync — Basics & the Trailing Slash

rsync — Flags That Matter

rsync — Remote Sync over SSH

Syncthing — CLI & Headless Setup

Proton Drive CLI — Auth & Filesystem

Proton Drive CLI — Sharing

Proton Pass CLI — Install, Login & Vaults

Proton Pass CLI — Items & Secrets

Proton Pass CLI — Run, Inject & SSH Agent

Things That Bite You

Five tools that all move or protect files, and five very different mental models. `rc`lone talks to 70+ cloud backends and treats them like a filesystem. `rsync` is the decades-old local/SSH workhorse everything else gets compared to. Syncthing is continuous peer-to-peer sync with no central server. Proton Drive CLI and Proton Pass CLI are both new (2025–2026) single-shot scripting tools built on Proton's end-to-end encryption — not background sync engines. Knowing which one to reach for saves more time than knowing any one of them deeply.

READ-ONLY / SAFE

MODIFIES DESTINATION

GOTCHA – READ THE NOTE

IRREVERSIBLE

PROTON TOOL

WHICH TOOL FOR THE JOB

DECISION GUIDE

| TOOL                    | BEST FOR                                                                                                                                             |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>rc</b> lone          | One-off or scheduled transfers to/from cloud storage (S3, Drive, Dropbox, 70+ backends). Also mounts cloud storage as a local filesystem.            |
| <b>rsync</b>            | Local-to-local or local-to-SSH-remote file sync. Fastest, most battle-tested delta-transfer tool for machines you control.                           |
| <b>Syncthing</b>        | Continuous two-way sync between your own devices, no cloud middleman, no central server. Set-and-forget, not scripted per-run.                       |
| <b>Proton Drive CLI</b> | Scripted, one-shot actions against Proton Drive — upload after a build, snapshot before an audit, revoke access. Not a background sync engine (yet). |
| <b>Proton Pass CLI</b>  | Pull secrets/credentials into scripts, CI/CD, and SSH workflows without ever putting them in plaintext files or shell history.                       |

# RCLONE — REMOTES & CONFIG

RCLONE

Rclone talks to a remote through a named config entry — set it up once, reference it by name everywhere else.

| COMMAND                                     | ACTION                                                                                                     |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>rclone config</code>                  | interactive config wizard — walks you through adding a remote                                              |
| <code>rclone config create name type</code> | create a remote non-interactively                                                                          |
| <code>rclone config show</code>             | show all configured remotes                                                                                |
| <code>rclone config file</code>             | show the config file path                                                                                  |
| <code>rclone listremotes</code>             | list all configured remote names                                                                           |
| <code>rclone obscure 'password'</code>      | obscure a password for use in the config file (not real encryption — just avoids storing it in plain text) |

# RCLONE – COPY, SYNC & MOVE

MODIFIES DESTINATION

`copy` only adds/updates — it never deletes. `sync` makes the destination an exact mirror, deleting anything not in the source. That distinction is the whole ballgame.

| COMMAND                                       | ACTION                                                            |
|-----------------------------------------------|-------------------------------------------------------------------|
| <code>rclone copy /local remote:backup</code> | local → cloud, additive only, never deletes at destination        |
| <code>rclone copy remote:backup /local</code> | cloud → local restore, additive only                              |
| <code>rclone sync /local remote:backup</code> | local → cloud MIRROR — deletes at destination what's gone locally |
| <code>rclone move /local remote:backup</code> | copy then delete from source once verified                        |
| <code>rclone check src dst</code>             | verify source and destination match, no transfer                  |
| <code>rclone --dry-run sync src dst</code>    | preview a sync with zero changes made — ALWAYS run this first     |



`sync` deletes at the destination. Always run with `--dry-run` first, or add `--max-delete N` so it aborts instead of silently wiping a destination that changed more than expected.

## Safer sync with a backup of anything it would delete/overwrite

```
rclone sync src dst --backup-dir remote:old-versions/$(date +%Y%m%d) --max-delete
```

## Common flags worth knowing

|                              |                                   |
|------------------------------|-----------------------------------|
| <code>--dry-run</code>       | # preview, no changes – use first |
| <code>-P / --progress</code> | # real-time transfer progress     |
| <code>--bwlimit 10M</code>   | # cap bandwidth                   |

```
--transfers 16 --checkers 32 --fast-list    # high-performance transfer
--include "*.jpg" --exclude "*"              # filter what moves
```

## RCLONE — BISYNC (TWO-WAY) ADVANCED — READ FIRST

Bidirectional sync between two paths, tracking changes on both sides between runs. Rclone's own docs call this an advanced command — read the limitations section before trusting it with anything you can't lose.

### First run establishes the baseline (required)

```
rclone bisync /local remote:path --resync
```

### Every run after that

```
rclone bisync /local remote:path
```



`--resync` only on the very first run (or deliberately, to re-baseline). Running it again treats one side as authoritative and can overwrite changes on the other. `--max-delete N` and `--check-access` are worth adding for anything important.

# RCLONE – MOUNT & FILTERS

MODIFIES LOCAL VIEW

| COMMAND                                                                | ACTION                                                       |
|------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>rclone mount remote:path /mnt/point --daemon</code>              | mount cloud storage as a local FUSE filesystem, backgrounded |
| <code>rclone mount remote:path /mnt/point --vfs-cache-mode full</code> | cache files locally for better performance / offline reads   |
| <code>fusermount -u /mnt/point</code>                                  | unmount (Linux)                                              |
| <code>umount /mnt/point</code>                                         | unmount (macOS)                                              |
| <code>rclone copy src dst --min-size 1M --max-size 1G</code>           | only transfer files in a size range                          |
| <code>rclone copy src dst --max-age 7d</code>                          | only files modified in the last 7 days                       |

# RSYNC — BASICS & THE TRAILING SLASH

GOTCHA

Same source-then-destination order as `cp`. The footgun is the trailing slash on the *source* — it changes whether the source directory itself gets copied, or just its contents.

## Trailing slash changes the result

```
rsync -av /src/folder /dst/      # copies folder itself -> /dst/folder/
rsync -av /src/folder/ /dst/     # copies folder's CONTENTS -> /dst/* (no wrapping)
```



Trailing slash on the source means "contents of"; no trailing slash means "the directory itself." This is the single most common rsync mistake, and it's silent — no error, just the wrong layout at the destination.

# RSYNC — FLAGS THAT MATTER

MODIFIES DESTINATION

| FLAG              | MEANING                                                                           |
|-------------------|-----------------------------------------------------------------------------------|
| -a, --archive     | archive mode — recursive, preserves perms/times/symlinks/owner (the daily driver) |
| -v, --verbose     | show what's happening                                                             |
| -z, --compress    | compress data during transfer (helps over slow links)                             |
| -P                | shorthand for --progress --partial (resumable, shows progress)                    |
| -n, --dry-run     | preview only — combine with -av to see what would happen                          |
| --delete          | delete files at destination that don't exist in source — makes it a real mirror   |
| -L, --copy-links  | follow symlinks and copy the target's data instead of the link                    |
| --exclude PATTERN | skip files/dirs matching a pattern (repeatable)                                   |



--delete turns rsync into a mirror, same risk profile as `rclone sync`. Always pair a first run with `-n` (dry-run) before adding `--delete` for real.

## The command you actually run most often

```
rsync -avP --delete src/ dst/
```



# RSYNC — REMOTE SYNC OVER SSH

MODIFIES DESTINATION

| COMMAND                                                                            | ACTION                                                                                              |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>rsync -avz -e ssh src/<br/>user@host:/dst/</code>                            | sync to a remote host over SSH                                                                      |
| <code>rsync -avz -e 'ssh -p 2222'<br/>src/ user@host:/dst/</code>                  | custom SSH port                                                                                     |
| <code>rsync -avz -e 'ssh -i<br/>~/.ssh/id_ed25519' src/<br/>user@host:/dst/</code> | specific SSH key                                                                                    |
| <code>rsync --link-dest=../previous<br/>/src/ /dst/current/</code>                 | hard-link unchanged files against a previous backup — classic space-saving incremental backup trick |

# SYNCTHING – CLI & HEADLESS SETUP

## CONTINUOUS P2P SYNC

Syncthing is a background daemon with a web GUI by default. On a headless box, everything under `syncthing cli` talks to the REST API of an already-running instance — start the daemon first.

| COMMAND                                                              | ACTION                                                                                       |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>syncthing</code>                                               | start the daemon (implicit 'serve'); prints the GUI/API URL on first run                     |
| <code>syncthing serve --no-browser --gui-address=0.0.0.0:8384</code> | headless-friendly start: don't launch a browser, bind GUI beyond localhost                   |
| <code>syncthing generate --gui-user=admin --gui-password=-</code>    | generate initial config + device ID without touching the REST API; reads password from stdin |
| <code>syncthing device-id</code>                                     | print this device's ID (needed to pair with another device)                                  |
| <code>syncthing paths</code>                                         | show config/data directory locations                                                         |
| <code>syncthing --version</code>                                     | print version info                                                                           |

## SYNCTHING CLI – TALKS TO A RUNNING INSTANCE'S REST API

| COMMAND                                                                                  | ACTION                              |
|------------------------------------------------------------------------------------------|-------------------------------------|
| <code>syncthing cli config folders list</code>                                           | list configured folders             |
| <code>syncthing cli config folders &lt;id&gt; devices add --device-id &lt;ID&gt;</code>  | share a folder with a paired device |
| <code>syncthing cli config devices add --device-id &lt;ID&gt; --name &lt;name&gt;</code> | add/pair a new device by its ID     |
| <code>syncthing cli show system</code>                                                   | system status, JSON output          |

| COMMAND                                       | ACTION                                |
|-----------------------------------------------|---------------------------------------|
| <code>syncthing cli operations restart</code> | restart the running instance          |
| <code>syncthing cli errors show</code>        | list pending errors needing attention |



Every `cli` subcommand has `--help`, and the tree is deep — `config`, `show`, `operations`, `errors`, and `debug` are the five top-level groups. Discover interactively rather than memorizing the whole hierarchy.



`syncthing debug reset-database` forces a full rescan/resync and must only be run while Syncthing is *not* running. Data isn't deleted, but every file gets rehashed — expensive on large folders.

# PROTON DRIVE CLI — AUTH & FILESYSTEM

## PROTON DRIVE CLI

A single binary (`proton-drive`), built on the same SDK as the official apps, for scripting Drive from cron, CI, and deploy pipelines. It's not a sync daemon — it runs one operation and exits. Sign-in happens through your browser; sessions are stored in your OS keychain, never a password on the command line.

| COMMAND                                                                                            | ACTION                                                                   |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>proton-drive auth login</code>                                                               | sign in via browser; session stored in OS keychain                       |
| <code>proton-drive filesystem list /my-files</code>                                                | list a folder                                                            |
| <code>proton-drive filesystem upload ./reports/* /my-files/Reports --conflict-strategy skip</code> | upload local files into a Drive folder                                   |
| <code>proton-drive filesystem download /my-files/Reports ./backups</code>                          | download a Drive folder to a local path                                  |
| <code>proton-drive version</code>                                                                  | confirm the installed build                                              |
| <code>proton-drive help</code>                                                                     | full command set; also <code>proton-drive &lt;command&gt; --help`</code> |

### Typical flow: upload a build artifact after CI finishes

```
proton-drive auth login
proton-drive filesystem upload ./dist/* /my-files/Releases/$(date +%Y%m%d) --con
```



Add `--json` (or `-j`) to any command for machine-readable output when you're piping into other tools.



Only the official app has a background sync engine. The CLI runs an operation and exits — good for "do this after the build finishes," wrong tool for "keep this folder continuously

mirrored." Use Syncthing or rclone bisync for that.

## PROTON DRIVE CLI – SHARING

PROTON DRIVE CLI

| COMMAND                                                                                                                     | ACTION                                      |
|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <code>proton-drive sharing status /my-files/Reports</code>                                                                  | see who currently has access                |
| <code>proton-drive sharing invite -<br/>-user person@pm.me --role<br/>editor /my-files/Reports</code>                       | invite someone with an editor role          |
| <code>proton-drive sharing invite -<br/>-user person@pm.me --role<br/>editor --message "text" /my-<br/>files/Reports</code> | same, with a message attached to the invite |

Fair use follows the same policy as the GUI apps: only upload/download what actually changed. Re-uploading unchanged files or rewriting whole folders on every run can get an account temporarily throttled.

# PROTON PASS CLI – INSTALL, LOGIN & VAULTS

## PROTON PASS CLI

Binary is `pass-cli`. Requires Pass Plus/Family/Professional or a bundle plan.

Everything routes through end-to-end encryption — this isn't a plaintext secrets store with a CLI bolted on.

| COMMAND                                                                       | ACTION                                                             |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>curl -fsSL https://proton.me/download/pass-cli/install.sh   bash</code> | install (macOS/Linux)                                              |
| <code>brew install protonpass/tap/pass-cli</code>                             | install via Homebrew                                               |
| <code>pass-cli login</code>                                                   | web-based login — prints a URL, completes in your browser          |
| <code>pass-cli login --interactive user@proton.me</code>                      | login entirely in the terminal (not for SSO/hardware-key accounts) |
| <code>pass-cli info</code>                                                    | verify current session                                             |
| <code>pass-cli logout</code>                                                  | end the session                                                    |

## VAULTS

| COMMAND                                                                                        | ACTION                                       |
|------------------------------------------------------------------------------------------------|----------------------------------------------|
| <code>pass-cli vault list</code>                                                               | list vaults you have access to               |
| <code>pass-cli vault create --name "Team Project"</code>                                       | create a vault                               |
| <code>pass-cli vault share --vault-name "Team Project" person@company.com --role editor</code> | share a vault (roles: viewer/editor/manager) |

| COMMAND                                                      | ACTION                              |
|--------------------------------------------------------------|-------------------------------------|
| <code>pass-cli vault delete -- vault-name "Old Vault"</code> | delete a vault AND everything in it |



`vault delete` is permanent and takes every item in the vault with it. No trash, no undo.

## AUTOMATION-FRIENDLY LOGIN (CI/CD)

**Personal access tokens — scoped, revocable, the right way to log in from a pipeline**

```
PROTON_PASS_PERSONAL_ACCESS_TOKEN="pst_xxxx...xxx::TOKENKEY" pass-cli login
```



Create the token first with `pass-cli pat create`, then grant it access to only the vaults/items it needs. Scoped tokens beat logging in with a full account session inside a pipeline.

# PROTON PASS CLI – ITEMS & SECRETS

## PROTON PASS CLI

| COMMAND                                                                                                            | ACTION                                        |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| <code>pass-cli item list --vault-name "Personal"</code>                                                            | list items in a vault                         |
| <code>pass-cli item create login --title "GitHub" --username u --generate-password --url https://github.com</code> | create a login item with a generated password |
| <code>pass-cli item view --vault-name "Personal" --item-title "GitHub"</code>                                      | view an item (aliases: get, show)             |
| <code>pass-cli item view "pass://Work/GitHub/password"</code>                                                      | view a single field via a pass:// URI         |
| <code>pass-cli item update --item-id ID --field "password=newvalue"</code>                                         | update one or more fields                     |
| <code>pass-cli item delete --share-id ID --item-id ID</code>                                                       | delete an item — permanent                    |



`item delete` is permanent, same as vault delete. No recovery.

## SECRET REFERENCES – THE PASS:// URI

Every field in Pass can be addressed directly without ever printing the value to your terminal history:

### Syntax

```
pass://<vault-identifier>/<item-identifier>/<field-name>
```

### Examples



```
pass://Work/GitHub/password  
pass://Personal/Email Login/username  
pass://Work/GitHub/totp          # current TOTP code  
pass://Work/GitHub/totp?totp=uri # raw otpauth:// URI instead
```

---

# PROTON PASS CLI – RUN, INJECT & SSH AGENT

## PROTON PASS CLI

This is the part that actually replaces "secrets sitting in a `.env` file" — reference them with `pass://`, let the CLI resolve at run time, and the plaintext never touches disk.

### Resolve `pass://` references from your environment and inject them into a command

```
export DB_PASSWORD='pass://Production/Database/password'
pass-cli run -- ./my-app
# app sees the real password; your shell history and .env file never do
```

### Same, but sourced from `.env` files (later files win)

```
pass-cli run --env-file base.env --env-file secrets.env -- node server.js
```



By default `run` masks secret values in stdout/stderr as `<concealed by Proton Pass>`. Use `--no-masking` only when you deliberately need to see them.

## SSH AGENT INTEGRATION

| COMMAND                                                                 | ACTION                                                     |
|-------------------------------------------------------------------------|------------------------------------------------------------|
| <code>pass-cli ssh-agent load</code>                                    | load Pass-stored SSH keys into your existing ssh-agent     |
| <code>pass-cli ssh-agent start</code>                                   | run Pass CLI itself as an SSH agent                        |
| <code>pass-cli ssh-agent daemon start</code>                            | run the agent detached, in the background, with a PID file |
| <code>pass-cli ssh-agent daemon status</code>                           | check whether the daemon is running                        |
| <code>pass-cli item create ssh-key generate --title "Deploy Key"</code> | generate a new SSH key stored directly in Pass             |

### Point SSH at the Pass agent

```
export SSH_AUTH_SOCK=$HOME/.ssh/proton-pass-agent.sock
```

---

# THINGS THAT BITE YOU

READ THIS ONCE



**rclone sync** and **rsync --delete** both mirror the destination. Anything at the destination that isn't in the source gets deleted. Dry-run first, every time, no exceptions.



**rsync's trailing slash on the source silently changes the result.** No error, just the wrong directory layout at the destination. Get in the habit of always deciding the slash on purpose.



**rclone bisync isn't "fire and forget" like Syncthing.** It's explicitly labeled advanced by its own docs; misusing **--resync** after the first run can overwrite the side you didn't mean to.



**Proton Drive CLI has no background sync engine.** It's a one-shot tool — great for a cron job or a deploy step, wrong tool if you wanted continuous mirroring. That's what the desktop app (or Syncthing/rclone) is for.



**Proton Pass personal access tokens beat full account logins in CI.** Scope a token to only the vaults a pipeline needs, not the whole account — if the pipeline leaks, the blast radius stays small.



**Never put a raw secret value in an environment variable you export normally.** Use **pass://** references with **pass-cli run** so the actual value only ever exists in the child process's memory, never in your shell history.



**Syncthing's headless setup needs the daemon running before cli commands work.** **syncthing cli ...** talks to a REST API — if nothing's serving that API yet, every cli subcommand just fails to connect.

